

Intelligent Data Processor Core Foundation

Suggested view for AI based engine for partner data processing:

Section I – Processing Engine Overview

Building an AI engine for processing partner data starting with ETL (Extract, Transform, Load) involves creating a system where an AI agent (or LLM) generates, executes, or manages the data pipelines. Instead of hardcoding scripts, we can build a system that parses prompts, maps schemas automatically, and writes its own transformation logic.

1. Architectural Overview

An AI-driven data processing engine relies on a three-tier architecture:

- **Orchestration Layer:** Traditional schedulers (like Apache Airflow) or workflow managers (like Kestra on [Shakudo](#)) manage the execution of tasks.
- **Agentic / LLM Layer:** The "Engine." This consists of an AI agent connected to data sources, equipped with APIs to generate SQL or Python code dynamically. This could be custom built using
- **Execution & Storage Layer:** The destinations and compute engines (e.g., Databricks, Snowflake, PySpark) where data is physically moved and processed.

2. Core Components to Build

To build the engine from scratch using frameworks like LangChain or LlamaIndex in Python, implement the following (please see section II for more insight on building AI engine foundation):

- **Dynamic Connection Tools:** Create tools for the AI agent to list available databases, fetch table schemas, and authenticate API connections safely (using tools to read the data specs).
- **Prompt Engine & Agents:** Design a system prompt that gives the AI an objective. Example: *"You are a data engineering assistant. Given the source schema X and destination schema Y, write the SQL or PySpark code to clean the data and load it into the warehouse."*

- **Enrichment Functions:** Use AI functions to parse messy, unstructured data. For example, instruct the LLM to run `ai_classify` or `ai_extract` on unstructured text (like support tickets or reviews) before loading the structured summary into the database.

3. Step-by-Step Implementation

We can construct a basic, functional AI ETL agent using Python, an LLM, and a framework like LangChain:

- **Step 1: Define the Source & Destination** Programmatically set up the API/JSON sources and database connections (e.g., PostgreSQL or Snowflake).
- **Step 2: Initialize the Agent Tools** Write Python functions (Tools) that allow the LLM to read schemas and execute Python/SQL code.
- **Step 3: Run the AI Agent** Pass the user prompt to the LLM. The agent will autonomously request the schema, figure out the column mappings, write the transformation logic, and load the data.
- **Step 4: Add Validation** Integrate a schema validator or a data quality checker to ensure the LLM's output conforms to the database constraints before committing the transaction.

4. Enterprise AI-Augmented ETL Tools

If building an entirely custom engine from scratch is too heavy an undertaking, most modern data platforms now offer built-in AI capabilities. We can leverage existing platforms to cut engineering time:

- **Databricks:** Use Lakeflow and Agentic AI functions (e.g., `ai_extract`, `ai_classify`) to embed models directly in the ETL workflows.
- **Matillion & Airbyte:** These platforms offer AI-assisted schema mapping and SQL optimization, which significantly reduce the manual lift of building pipelines.
- **Informatica:** Offers AI-powered mapping recommendations and conversational Natural Language interfaces for pipeline generation.

If we are looking to build a custom solution, it helps to know:

- What **frameworks** (e.g., LangChain, LlamaIndex) do we prefer to use?
- What are the primary **data sources** (e.g., APIs, local files, relational databases)?

We can discuss to outline the exact **code structure or tool definitions** as needed.

Section II – Processing Engine Overview

To build a custom AI engine generally either a foundational machine learning framework (like PyTorch or TensorFlow) or a self-hosted inference engine (like Ollama or vLLM) that lets you run large language models (LLMs) locally on your own hardware, ensuring data privacy and total customization.

Top Open-Source AI Engines and Frameworks

1. Local LLM Execution & APIs

- **Ollama:** The easiest way to run and serve open-source LLMs locally (e.g., Llama 3, Mistral) on your machine. It bundles model weights and configurations into a single package.
- **vLLM:** A high-throughput and memory-efficient LLM serving engine. It is highly optimized for deploying models in production environments and scales well on GPUs. Access the project repository on the vLLM GitHub.
- **LocalAI:** A self-hosted, drop-in replacement for the OpenAI API that allows you to run local LLMs, generate images, and transcribe audio on consumer-grade hardware without a GPU. Find it on the LocalAI Documentation Hub.

2. Foundational ML & Deep Learning Frameworks

- **PyTorch:** A highly popular, Python-based deep learning framework utilized extensively for AI research, rapid prototyping, and dynamic model training. Learn more on the PyTorch Official Site.
- **TensorFlow:** An end-to-end open-source platform developed by Google, suited for numerical computation and deploying machine learning models across web, mobile, and edge devices. Explore capabilities via the TensorFlow Official Site.
- **Keras:** A high-level neural networks API written in Python that is highly approachable for beginners and can run on top of TensorFlow, PyTorch, or JAX.